به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین اول

مدرس: دکتر مهدی صفرنژاد — انتشار: ۱۱ مهر ۱۴۰۰

مقدمه و تحلیل الگوریتم‌ها

سؤالات را با دقت بخوانید و روی همه آن‌ها وقت بگذارید. تمرین‌های تئوری تحویل گرفته نمی‌شوند اما از آن‌ها سؤالات کوییز مشخص می‌شود. بنابراین روی سؤالات به خوبی فکر کنید و در کلاس‌های حل تمرین مربوطه شرکت کنید.

سؤال ۱. با مثال نقض، نادرستی گزاره‌های زیر را نشان دهید.

آ. توابع $f : \mathbb{N} \to \mathbb{N}$ و $g : \mathbb{N} \to \mathbb{N}$ وجود ندارند که $f(n) + g(n) \neq \mathcal{O}(f(n))$ و $f(n) + g(n) \neq \mathcal{O}(g(n))$ باشد.

ب. به ازای هر تابع $f : \mathbb{N} \to \mathbb{N}$، آن‌گاه $f(n) = \Theta(f(\frac{n}{2}))$.

پ. به ازای توابع $f_i : \mathbb{N} \to \mathbb{N}$، آن‌گاه $\sum_i f_i(n) = \mathcal{O}\left(\max_i\{f_i(n)\}\right)$.

پاسخ: آ. گزاره نادرست است، زیرا ابتدا فرض کنید $f(n) = \left\{ \begin{array}{ll} 1, & n = 2k+1 \\ n, & n = 2k \end{array} \right\}$ و $g(n) = \left\{ \begin{array}{ll} n, & n = 2k+1 \\ 1, & n = 2k \end{array} \right\}$ باشد. هم‌چنین فرض کنید

$$h(n) = f(n) + g(n)$$

در نتیجه

$$h(n) = n + 1$$

باشد، واضح است که

$$h(n) \neq \mathcal{O}(f(n)) \wedge h(n) \neq \mathcal{O}(g(n)).$$

در واقع به ازای هر n_0 و c، $n \geq n_0$ وجود دارد که $h(n) > cf(n)$ و $h(n) > cg(n)$.

ب. گزاره نادرست است، زیرا فرض کنید $f(n) = 2^n$ باشد آن‌گاه $f(\frac{n}{2}) = \sqrt{2}^n$ خواهد بود که بیان‌گر این می‌باشد که $f(n) \neq \Theta(f(\frac{n}{2}))$.

پ. گزاره نادرست است، زیرا فرض کنید

$$\forall i \quad f_i = 1$$

باشد، واضح است که

$$\sum_i f_i = n \neq \mathcal{O}(1).$$

سؤال ۲. پیچیدگی زمانی قطعه کد های زیر را بیابید.

آ.
```
for (int i = 0; i < n; i++) {
    for (int j = n; j > i; j--) {
        //O(1)
    }
}
```

ب.
```
for (int i = n/2; i <= n; i++) {
    for (int j = 2; j <= n; j *= 2) {
        //O(1)
    }
}
```

پ.
```
int j = 1;
int i = 0;
while (i < n) {
    //O(1)
    i += j;
    j++;
}
```

ت.
```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j < n; j+=i) {
      //O(1)
    }
}
```

ث.
```
for (int i = n; i > 0; i/=2) {
    for (int j = 0; j < i; j++) {
        //O(1)
    }
}
```

پاسخ:

آ. پاسخ : $O(n*n)$

راه حل: $n+n-1+\ldots+2+1=\frac{n(n-1)}{2}=O(n)$

ب.

پاسخ: $O(n\ logn)$

راه حل:

حلقه اول $n/2$ بار و حلقه درونی $logn$ بار اجرا می شود

پ.

پاسخ:$O(\sqrt{n})$

راه حل: i در هر مرحله جمع اعداد ۱ تا j است اگر j تا k پیش برود (برنامه k بار اجرا می شود)

$$1+2+\ldots+k-1+k=\frac{k\left(k+1\right)}{2}$$

و حلقه زمانی پایان میاید که :

$$\frac{k\left(k+1\right)}{2}>n$$

در نتیجه

$$k=O\left(\sqrt{n}\right).$$

ت.

پاسخ: $O(n\ logn)$

راه حل: حلقه در مرحله اول n بار، در مرحله دوم n/۲ بار و ... اجرا می شود

پس داریم:

$$O\left(s\right)=O\left(n+\frac{n}{2}+\frac{n}{3}+\frac{n}{4}+\ldots+\frac{n}{n}\right)=O(n\left(1+\frac{1}{2}+\frac{1}{3}+\ldots+\frac{1}{n}\right)=O(n\left(\sum_{i=1}^{n}\frac{1}{i}\right))$$

حال $O\left(\sum_{i=1}^{n}\frac{1}{i}\right)$ را حساب می کنیم:

$$\sum_{i=1}^{n} \frac{1}{i} = \int_{1}^{n} \frac{1}{x}\, dx = \ln(n)$$

$$\sum_{x=1}^{n} \frac{1}{x+1} \leq \int_{1}^{n} \frac{1}{x}\, dx = \ln n \leq \sum_{x=1}^{n} \frac{1}{x}$$

$$O\left(\frac{1}{1} + \frac{1}{2} + \ldots + \frac{1}{n}\right) = O(\log n).$$

ث.

پاسخ : $O(n)$
راه حل:

$$O\left(n\left(1 + \frac{1}{2} + \frac{1}{4} + \ldots\right)\right) = O(2n) = O(n).$$

سؤال ۳. با استفاده از روش های دلخواه پیچیدگی زمانی رابطه های بازگشتی زیر را پیدا کنید.

آ. $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log(n)$
ب. $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$
پ. $T(n) = \left(\frac{2}{n}\right)(T(0) + \ldots + T(n-1)) + c$

پاسخ:

آ. از تغییر متغیر $m = \log(n)$ استفاده می کنیم:

$$T(2^m) = 2T\left(2^{\frac{m}{2}}\right) + m.$$

سپس قرار میدهیم $T(2^m) = S(m)$
در نتیجه:

$$S(m) = 2S(\frac{m}{2}) + m$$

طبق قضیه اصلی:

$$S(m) = T(2^m) = m\log(m)$$

$$T(n) = \log(n)\log(\log(n))$$

ب. اگر درخت مربوط به $T(n)$ را رسم کنیم، بلندترین شاخه دارای ارتفاع $log_{\frac{3}{2}}^{n}$ و کوتاه ترین شاخه دارای ارتفاع log_{3}^{n} است.
در نتیجه:

$$T(n) = O\left(nlog_{\frac{3}{2}}^{n}\right) = O\left(log_{\frac{3}{2}}^{3} \times nlog_{3}^{n}\right) = O(nlog(n))$$

$$T(n) = \Omega\left(nlog_{3}^{n}\right) = \Omega(nlog(n))$$

در نتیجه:

$$T(n) = \theta(nlog(n)).$$

سؤال ۴. رابطه ی بازگشتی مربوط به تیکه کد زیر را پیدا کنید و سپس با روشی دلخواه پیچیدگی زمانی آن را بدست آورید.

```
1
2  int gcd(int a, int b)
3  {
4      if(a == b)
5                  return a;
6      if(a > b)
7                  gcd(a % b, b);
8      else
9                  gcd(a, b % a);
10 }
```

پاسخ:
در هر مرحله تابع gcd یکی ازآرگمان ها را نصف می کند(حداکثر). به طور مثال:
اگر $b < a/2$ باشد در مرحله بعد داریم $a' = b$ و $b' < a/2$ چرا که % به اندازه b یا بیشتر از آن از a کم می کند
اگر $b < a/2$ باشد در مرحله بعد داریم $a' = b$ و $b' < a/2$ چرا که % حداکثر $b-1$ را برمیگرداند
در نتیجه در هر مرحله‌ي بازگشتی تابع gcd حداکثر یکی از عبارت ها را نصف می‌کند. در نتیجه پیچیدگی زمانی این قطعه کد برابر با $O(log(n))$ که در این جا n ماکسیمم a یا b است.
اگر $g(x,y) \leq T(n)$ در صورتی که $n = xyd = \frac{x}{x \bmod y}$ باشد:

$$T(n) = T\left(\frac{n}{d}\right) + c$$

$$T(n) = T\left(\frac{n}{d^2}\right) + 2c$$

$$T(n) = T\left(\frac{n}{d^k}\right) + kc$$

$$\text{When } \frac{n}{d^k} = 1 \implies n = d^k \implies k = \log_d^n$$

$$T(n) = T(1) + c\log_d^n$$

$$T(xy) = 1 + c\log_d^{xy}$$

$$\implies g(x,y) \leq T(xy)$$

$$\implies g(x,y)\ \epsilon\ O(log\ d(xy)).$$

سؤال ۵. می‌خواهیم در رشته ای از حروف کوچک انگلیسی، بلندترین زیر رشته‌ای که در آن هیچ حرفی دو بار تکرار نشده است را پیدا کنیم. شبه‌کدی بنویسید که این کار را در مرتبه $O(n)$ انجام دهد.(برای راحتی می‌توانید فرض کنید که رشته به صورت آرایه‌ای از اعداد ۰ تا ۲۵ به شما داده شده است)

پاسخ:

سؤال ۶. آرایه $w[1..n]$ داده شده است که حاوی n عدد طبیعی است. $idx[i]$ را این‌گونه تعریف می‌کنیم:

$$idx[i] = \max\{j : 1 \leq j < i \ \wedge \ w[j] \leq w[i]\}.$$

الگوریتم زیر را برای پیدا کردن idx اجرا خواهیم کرد:

آ. نشان دهید الگوریتم بالا خاتمه‌پذیر است.

ب. نشان دهید الگوریتم پاسخ صحیح را تولید می‌کند.

پ. مرتبه زمانی شبه‌کد را پیدا کنید. (راهنمایی: تعداد دفعات رجوع به هر خانه آرایه محاسبه نمایید.)

پاسخ:

آ. الگوریتم خاتمه‌پذیر است، زیرا در هر بار اجرای حلقه داخلی، مقدار شمارنده j مقداری نزولی نسبت به حالت قبلی خواهد داشت و در نتیجه حلقه داخلی با شمارنده j در ابتدا از $i-1$ شروع خواهد شد و در بدترین حالت هر بار یک واحد از آن کم خواهد شد تا به مقدار -1 برسد و حلقه داخلی خاتمه یابد. از طرفی حلقه بیرونی نیز هر بار یک واحد افزایش می‌یابد تا به n برسد. در نتیجه الگوریتم حتماً خاتمه پذیر خواهد بود.

Algorithm 1 finding the length of longest substring with unique characters

```
1:  procedure LONGESTUNIQUESUBSTTR(str[])
2:      result ← 0, i ← 0
3:
4:      lastIndex ← int[26]
5:
6:      while i < 26 do
7:
8:          lastIndex[i] ← −1
9:
10:         i+ = 1
11:
12:     i ← 0, j ← 0
13:     while i < str.length do
14:         j ← j > lastIndex[str[i]] + 1?j : lastIndex[str[j]] + 1
15:         result ← result > i − j + 1?result : i − j + 1
16:         lastIndex[str[i]] ← i
17:         i+ = 1
18:     return result
```

Algorithm 2 Checking correctness and termination

```
1:  procedure ALG(w[1..n], idx[1..n])
2:      idx[1] = −1
3:      for i=2 to n do
4:          j = i − 1
5:          while w[j] > w[i] ∧ j ≠ −1 do
6:              j = idx[j]
7:          idx[i] = j
```

ب. هدف مسئله، این است که به ازای هر i، مقدار $idx[i]$ برابر با بزرگ‌ترین jای باشد که $j < i$ باشد و $w[j] \leq w[i]$ باشد. از طرفی مطابق الگوریتم، حلقه داخلی نیز ابتدا از $i - 1$ شروع کرده و هر بار مقداری کاهشی نسبت به مقدار قبلی خواهد داشت. با توجه به شرط مسئله، اولین j که شرط مسئله را ارضا کند به عنوان $idx[i]$ شناخته خواهد شد که درست است. زیرا مقدار آن بیشینه است و همچنین شرط $w[j] \leq w[i]$ نیز برقرار است. پس الگوریتم به درستی کار می‌کند.

پ. برای یافتن مرتبه زمانی شبه‌کد این‌گونه عمل خواهیم کرد، با به توجه حلقه‌ها، الگوریتم به هر خانه آرایه idx حداکثر دو بار مراجعه خواهد کرد، بار اول برای یافتن j مورد نظر و بار دوم نیز هنگامی خواهد بود که در گام‌های بعدی حلقه ممکن است به آن خانه آرایه idx مراجعه کند. در نظر داشته باشید هنگامی که در اجرای kام که $i < k$ است، به آرایه $idx[i]$ مراجعه صورت گیرد،در اجراهای بعدی یعنی اجرای $k < t$ به $idx[i]$ مراجعه‌ای صورت نخواهد گرفت و ممکن است به خانه‌ای قبل از $idx[i]$ مراجعه صورت گیرد، زیرا در درون حلقه، مقدار شمارنده $j = idx[k]$ خواهد شد. پس در نتیجه به هر خانه آرایه idx به تعداد بار ثابتی مراجعه می‌شود، از طرفی، طول آرایه idx برابر با n است، پس مرتبه شبه کد بالا $\mathcal{O}(n)$ است.

موفق باشید